BÀI THỰC HÀNH CHƯƠNG 4

I. Tạo Form dùng ngôn ngữ HTML

Bài này sẽ thực hành tạo form bằng cách nhập mã HTML thủ công. Qua đây cũng cho phép ta so sánh cách thức kiểm tra tính hợp lệ và gửi các trường thông tin khác nhau. Bài này sẽ thực hành trong một dự án Django mới để tránh ảnh hưởng đến dự án **Bookr**.

1. Tạo mới dự án Django, dự án này có thể sử dụng chung môi trường ảo với dự án **Bookr**. Mở cửa sổ **terminal** mới và kích hoạt môi trường ảo sau đó sử dụng **django-admin** để tạo dự án có tên là **form_project** bằng lệnh sau:

   ```
   django-admin startproject form_project
   ```

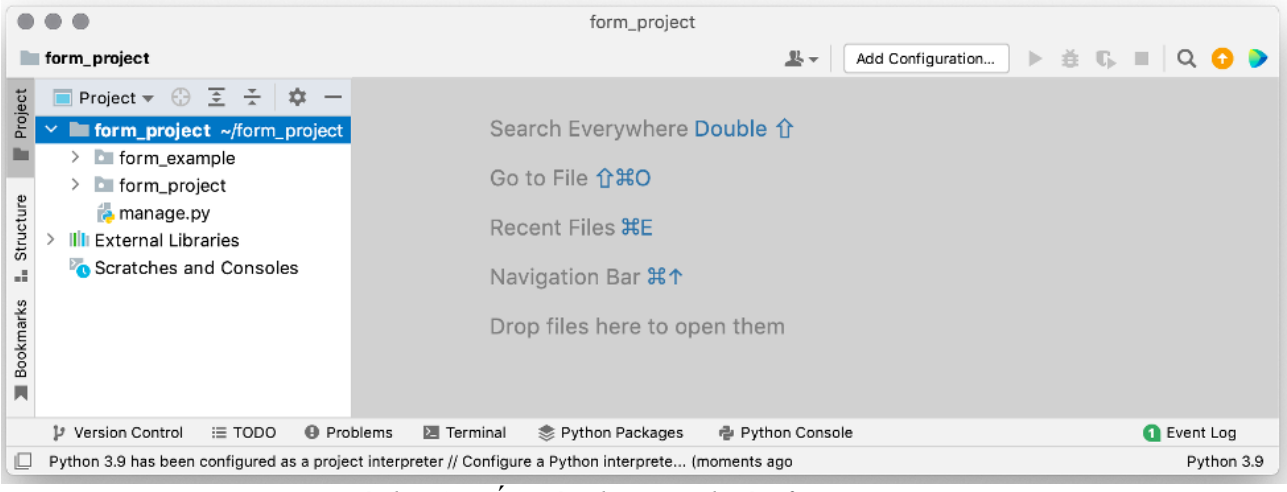
   Lệnh này sẽ tạo dự án Django trong thưc mục form_example.

2. Tạo mới một app trong dự án này sử dụng lệnh **startapp**. Đặt tên app là **form_example**. Lưu ý dùng lệnh **cd** để chuyển vào thư mục dự án **form_project** trước khi chạy lệnh sau:

   ```
   python manage.py startapp form_example
   ```

   Lệnh này sẽ tạo thưc mục cho app **form_example** trong thư mục **form_project**.

3. Mở thư mục dự án trên **PyCharm**, đảm bảo cấu trúc thư mục dự án như trong **Hình I-1**.



Hình I-1. Cấu trúc thư mục dự án form_project

4. Tạo một bộ cấu hình để chạy trình chủ web với lệnh **runserver** như theo hướng dẫn ở Bài thực hành **III-B** ở **Chương 1**.
5. Mở tập tin **settings.py** trong thưc mục **form_project** và thêm app **form_example** vào mục **INSTALLED_APPS**.
6. Tạo một thư mục chứa các Template cho app **form_example** bằng cách bấm phải chuột vào thư mục form_example rồi chọn **New → Directory**, nhập tên thư mục là **templates**.
7. Ta cần một template HTML để hiển thị form. Tạo template này bằng cách bấm phải chuột vào thư mục **templates** vừa tạo rồi chọn **New → HTML File**, nhập tên tập tin là **form-example.html**.
8. Sau khi tạo, tập tin **form-example.html** sẽ được tự động mở trong **PyCharm**. Ta bắt đầu thực hiện tạo form và các thành phần nhập liệu. Tạo form với phương thức gửi dữ liệu **POST** bằng thẻ <form> như sau:

```
<form method="post">
</form>
```

Cặp thẻ này được đặt giữa cặp thẻ **<body>** và **</body>**. Hiện tại, thuộc tính **action** của form không có, điều này có nghĩa là dữ liệu của form sẽ được gửi lại đến chính trang hiện tại (hay trang có cùng URL với trang hiện tại).

9. Thêm một số hộp nhập văn bản bằng cách chèn đoạn mã sau vào giữa cặp thẻ **<form>:**

```
<p>
    <label for="id_text_input">Text Input</label><br>
    <input id="id_text_input" type="text" name="text_input"
value="" placeholder="Enter some text">
</p>
<p>
    <label for="id_password_input">Password Input</label><br>
    <input id="id_password_input" type="password"
name="password_input" value="" placeholder="Your password">
</p>
```

Lưu ý: Để thêm khoảng trống giữa các hộp nhập ta sử dụng thêm thẻ **<p>.**

10. Tiếp theo, ta thêm **2 hộp kiểm (checkbox)** và **3 nút chọn (radio button)** bằng cách thêm đoạn mã sau vào sau đoạn mã ở bước trước, đoạn mã này cần nằm trước thẻ **</form>:**

```
<p>
    <input id="id_checkbox_input" type="checkbox"
name="checkbox_on" value="Checkbox Checked"
checked>
    <label for="id_checkbox_input">Checkbox</label>
</p>
<p>
    <input id="id_radio_one_input" type="radio"
name="radio_input" value="Value One">
    <label for="id_radio_one_input">Value One</label>
    <input id="id_radio_two_input" type="radio"
name="radio_input" value="Value Two" checked>
    <label for="id_radio_two_input">Value Two</label>
    <input id="id_radio_three_input" type="radio"
name="radio_input" value="Value Three">
    <label for="id_radio_three_input">Value Three</label>
</p>
```

11. Tiếp đến là **hộp chọn sổ xuống** (**drop-down listbox**) để chọn một cuốn sách yêu thích nhất. Thêm đoạn mã sau vào sau đoạn mã ở bước trước:

```
<p>
    <label for="id_favorite_book">Favorite Book</label><br>
    <select id="id_favorite_book" name="favorite_book">
        <optgroup label="Non-Fiction">
            <option value="1">Deep Learning with Keras</option>
            <option value="2">Web Development with
Django</option>
        </optgroup>
```

```
        <optgroup label="Fiction">
            <option value="3">Brave New World</option>
            <option value="4">The Great Gatsby</option>
        </optgroup>
    </select>
</p>
```

Ở đây 4 nút chọn sẽ chia thành 2 nhóm. Người dùng chỉ có thể chọn 1 trong 4 tùy chọn này.

12. Tiếp theo là **hộp danh sách nhiều lựa chọn (multiple select),** bằng cách sử dụng thuộc tính **multiple**. Thêm đoạn mã sau vào sau đoạn mã ở bước trước:

```
<p>
    <label for="id_books_you_own">Books You Own</label><br>
    <select id="id_books_you_own" name="books_you_own" multiple>
        <optgroup label="Non-Fiction">
            <option value="1">Deep Learning with Keras</option>
            <option value="2">Web Development with
Django</option>
        </optgroup>
        <optgroup label="Fiction">
            <option value="3">Brave New World</option>
            <option value="4">The Great Gatsby</option>
        </optgroup>
    </select>
</p>
```

Ở đây, người dùng có thể chọn tối thiểu 0 và tối đa 4 lựa chọn trong danh sách.

13. Tiếp theo, ta thêm một **hộp nhập văn bản dài (textarea)** bằng cách thêm đoạn mã sau:

```
<p>
    <label for="id_text_area">Text Area</label><br>
    <textarea name="text_area" id="id_text_area"
placeholder="Enter multiple lines of text"></textarea>
</p>
```

14. Đối với các hộp nhập dữ liệu kiểu đặc biệt khác như số, địa chỉ email, ngày tháng,… Ta thêm các đoạn mã sau:

```
<p>
    <label for="id_number_input">Number Input</label><br>
    <input id="id_number_input" type="number"
name="number_input" value="" step="any" placeholder="A number">
</p>
<p>
    <label for="id_email_input">Email Input</label><br>
    <input id="id_email_input" type="email" name="email_input"
value="" placeholder="Your email address">
</p>
<p>
    <label for="id_date_input">Date Input</label><br>
```

```
    <input id="id_date_input" type="date" name="date_input"
value="2019-11-23">
</p>
```

15. Tiếp theo, thêm các **nút bấm** để gửi dữ liệu của form lên máy chủ web (**submit**) bằng cách bổ sung đoạn mã lệnh sau vào sau đoạn mã ở bước trước:

```
<p>
    <input type="submit" name="submit_input" value="Submit
Input">
</p>
<p>
    <button type="submit" name="button_element" value="Button
Element">Button With<strong> Styled</strong> Text</button>
</p>
```

Phần này đưa ra 2 cách thể thêm nút **submit**, thực tế ta chỉ cần sử dụng một trong hai nút trên là đủ.

16. Cuối cùng, thêm một trường ẩn bằng cách thêm đoạn mã sau:

```
<input type="hidden" name="hidden_input" value="Hidden Value">
```

Trường ẩn này sẽ không hiển thị trên giao diện của form nên không thể chỉnh sửa thủ công.

Sau khi nhập xong các đoạn mã trên ta lưu tập tin **form-example.html** lại.

17. Thực hiện cập nhật lại tập tin **views.py** của app **form_example** để hiển thị form. Ta thêm một hàm mới là **form_example** để hiển thị trang **form-example.html** bằng đoạn mã sau:

```
def form_example(request):
    return render(request, "form-example.html")
```

Lưu và đóng tập tin **views.py**.

18. Thêm **một ánh xạ URL** để gọi đến hàm form_example bằng cách thêm đoạn mã sau vào phần khai báo biến **urlpatterns** của tập tin **urls.py:**

```
path('form-example/', form_example.views.form_example)
```

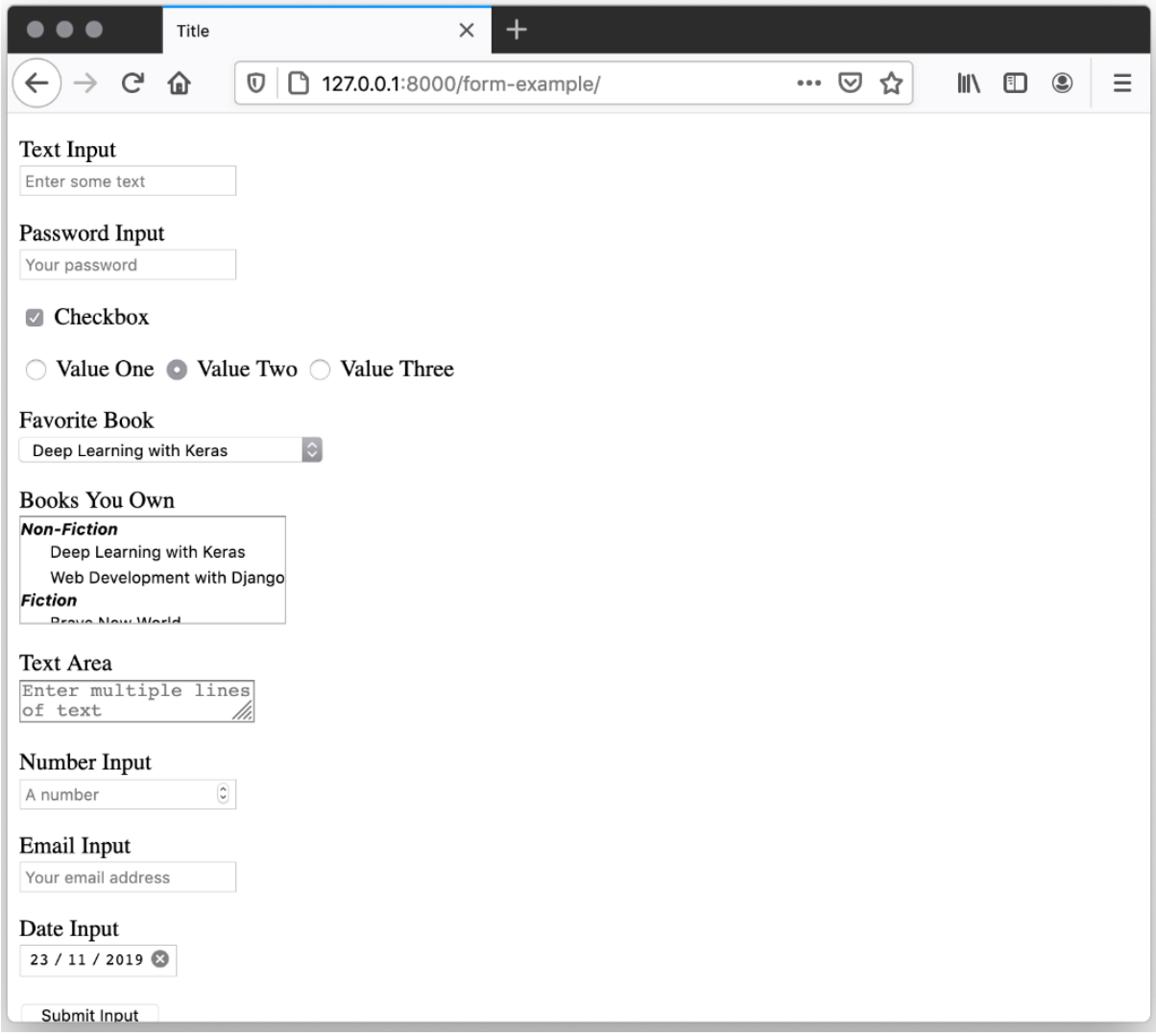
Lưu ý: **import** tập tin **form_example.views** trước phần khai báo **urlpatterns**.

Lưu và đóng tập tin **urls.py** lại.

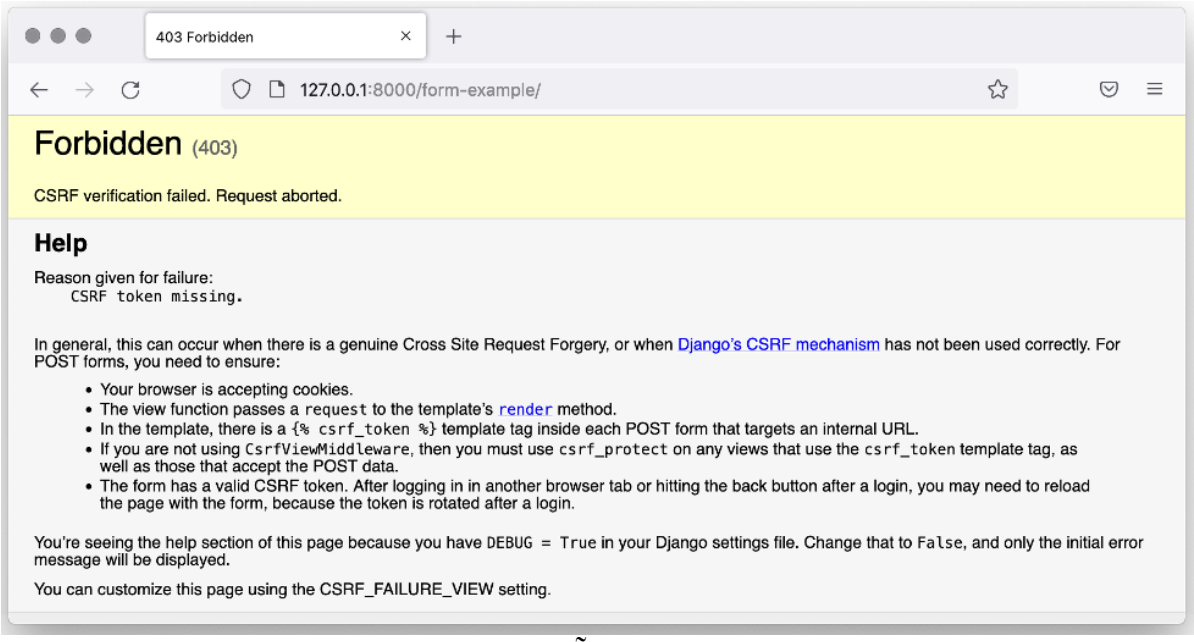
19. Khởi động máy chủ phát triển Django và truy cập tến view vừa tạo bằng URL sau: http://127.0.0.1:8000/form-example/. Kết quả giao diện trang web sẽ như **Hình I-2.** Thử nhập văn bản không hợp lệ vào các hộp nhập số, email và ngày tháng sau đó nhấn nút **Submit**. Công cụ kiểm tra tính hợp lệ của dữ liệu sẵn có của HTML sẽ ngăn gửi dữ liệu đi và hiện thông báo dữ liệu không hợp lệ.

Sau khi sửa các phần dữ liệu nhập vào không hợp lệ và bấm nút **Submit** thì dữ liệu sẽ được gửi đến máy chủ web. Tuy nhiên, ta sẽ nhận về một thông báo lỗi **CSRF verification failed** như trong **Hình I-3**. Sinh viên có thể đọc về tính năng bảo mật này của Django ở **phần 4.1** trong **slide bài giảng** và ở **trang 273-275 của Giáo trình.**

20. Khi nhận thông báo lỗi **CSRF verification error,** ta có thể bấm nút **Back** trên thanh công cụ của trình duyệt web để trở về trang web chứa form nhập liệu.



Hình I-2. Giao diện của trang form-example



Hình I-3. Thông báo lỗi CSRF verification error

II. Làm việc với dữ liệu gửi từ Form

Phần này sẽ thực hành một số thao tác trên dữ liệu gửi đến máy chủ web từ Form nhập liệu và kiểm tra dữ liệu đã gửi đến máy chủ web.

1. Trước tiên, mở tập tin **views.py** của app **form_example** và điều chỉnh hàm **form_example** để in tất cả giá trị có trong thông điệp **request.POST** trên giao diện dòng lệnh bằng đoạn mã sau:

```
for name in request.POST:
    print("{}: {}".format(name, request.POST.getlist(name)))
```

   Đoạn mã này sẽ duyệt qua tất cả các khóa của thông điệp **request.POST** và in khóa kèm giá trị tương ứng ra. Thông điệp **POST** này có kiểu dữ liệu là **QueryDict** nên có thể có nhiều giá trị tương ứng với 1 khóa, vì vậy cần dùng phương thức **getlist()** để lấy ra tất cả giá trị.

2. Truyền giá trị của **phương thức gửi dữ liệu** (**method**) qua một biến ngữ cảnh tên là **method** của hàm **render().** Đoạn mã gọi hàm **render()** sẽ sửa lại như sau:

```
return render(request, "form-example.html", {"method":
request.method})
```

3. Ta sẽ hiển thị giá trị của biến method trên trang template bằng cách thêm đoạn mã sau vào tập tin **form-example.html**. Phần thêm này sẽ nằm ngay ở trên đầu trang web nên ta đặt ngay sau tag **<body>.**

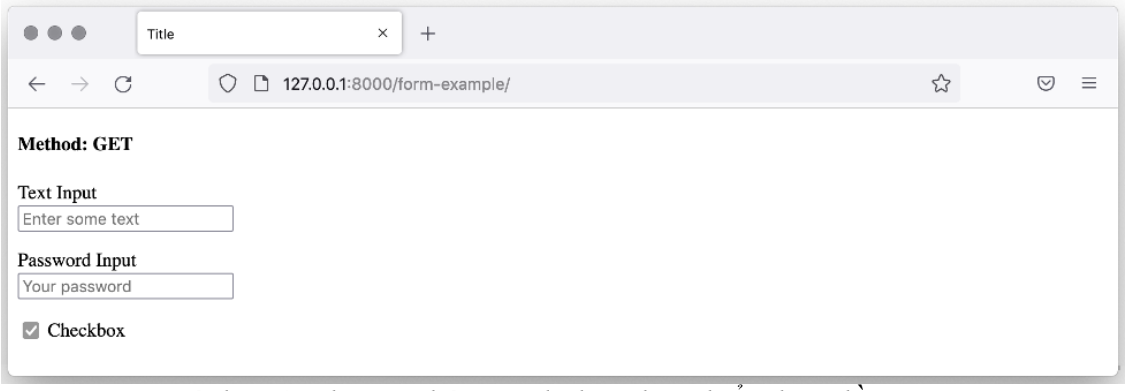
```
<body>
    <h4>Method: {{ method }}</h4>
```

   Lưu ý là ta có thể truy cập đến loại phương thức gửi dữ liệu (**method**) trực tiếp từ biến **request** trong template mà không cần truyền qua biến ngữ cảnh. Ở đây ta chỉ thử truy cập đến biến ngữ cảnh method này, ở phần sau ta sẽ thay đổi hành vi của trang web dựa vào biến **method** này.

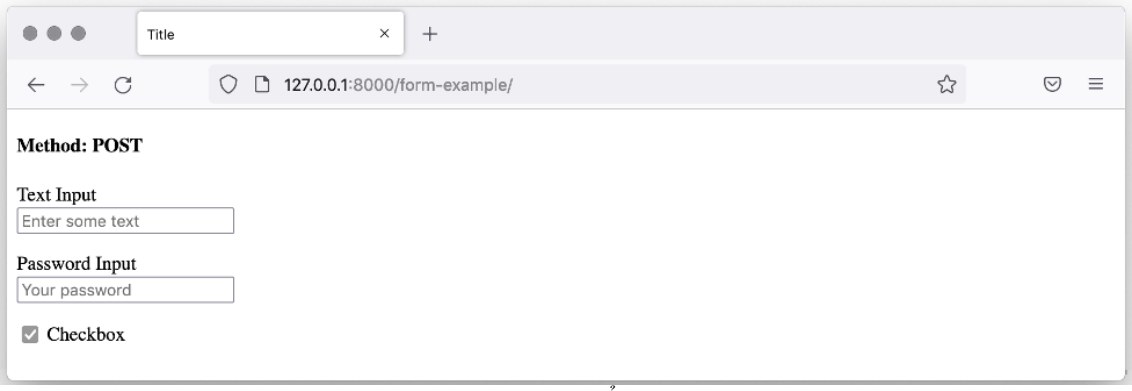
4. Ta cũng cần thêm **CSRF token** vào Form HTML để tránh gặp lỗi **CSRF verification failed** bằng cách đặt tag template **`{% csrf_token %}`** sau tag mở form **<form>.** Phần đầu của đoạn mã định nghĩa form bây giờ sẽ có dạng như sau:

```
<form method="post">
    {% csrf_token %}
```

5. Khởi chạy máy chủ phát triển Django và mở trang **form-example** với URL sau http://127.0.0.1:8000/form-example/, ta sẽ nhận được phương thức gửi dữ liệu là **GET** hiển thị trên đầu trang web sẽ như **Hình II-1**.
6. Nhập một đoạn văn bản hoặc dữ liệu vào mỗi hộp nhập và submit form bằng cách bấm vào nút **Submit Input**. Ta sẽ thấy dữ liệu được gửi đến máy chủ và trang web được tải lại với phương thức gửi dữ liệu bây giờ đổi thành **POST** như **Hình II-2**.
7. Trở lại cửa sổ **PyCharm** và xem khung giao diện dòng lệnh ở phần dưới cửa sổ. Nếu không có thì bấm vào nút **Run** như **Hình II-3** (nếu đang chạy máy chủ phát triển ở chế độ **Debug** thì bấm vào nút **Debug** thay vì nút **Run**). Kết quả phần giao diện dòng lệnh sẽ liệt kê các giá trị có trong biến **request.POST** như **Hình II-4.**

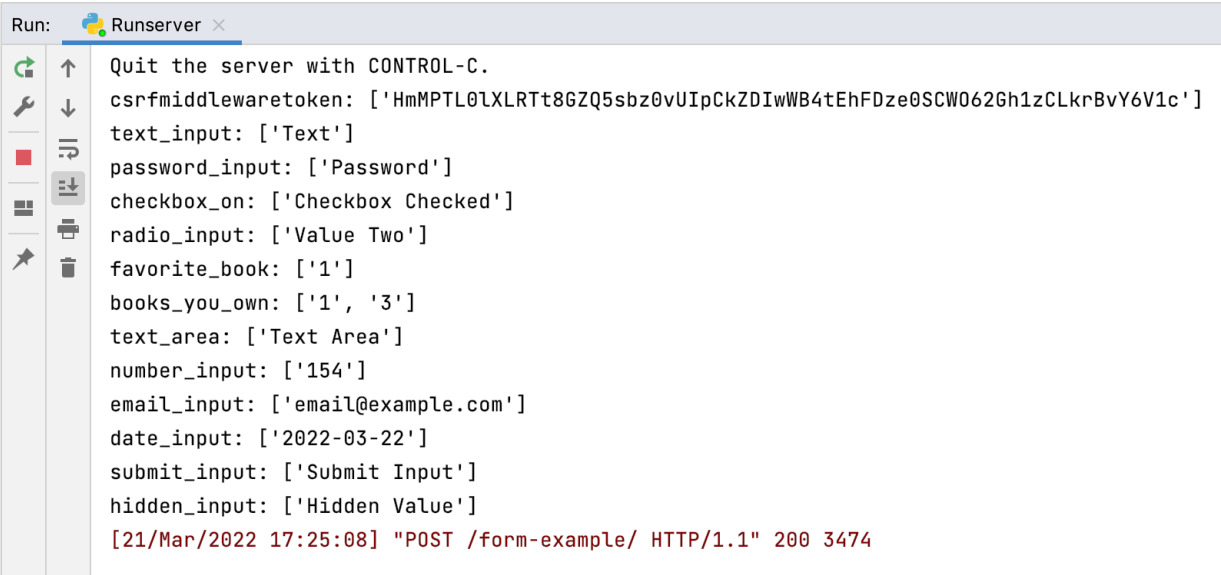


Hình II-1. Phương thức gửi dữ liệu được hiển thị ở đầu trang



Hình II-2. Phương thức gửi dữ liệu đã đổi thành POST sau khi submit form

Hình II-3. Nút Run ở phần dưới của cửa sổ ứng dụng PyCharm



Hình II-4. Các giá trị nhập vào của form gửi đến máy chủ web được liệt kê trong khung Run

Một số lưu ý từ kết quả của giao diện dòng lệnh tại Hình II-4:

- Tất cả các giá trị trong form gửi đến máy chủ web đều ở dạng **văn bản**, kể cả các giá trị nhập là kiểu số hay ngày tháng.

- Đối với dữ liệu nhập từ các hộp chọn, giá trị trong thuộc tính **value** của các hộp được chọn sẽ được gửi đi chứ không phải là nội dung văn bản của tag **<option>.**
- Nếu ta chọn nhiều tùy chọn cho trường thông tin **book_you_own**, ta sẽ thấy nhiều giá trị tương ứng với khóa này. Đây là lý do ta phải sử dụng phương thức **getlist()** như đã đề cập ở **bước 1.**
- Nếu một **hộp kiểm (checkbox)** được chọn, ta sẽ nhận được giá trị là **checkbox_on** như trong giao diện dòng lệnh. Nếu hộp kiểm đó không được chọn thì khóa **checkbox_on** sẽ không xuất hiện trong dữ liệu gửi đi.
- Ta cũng thấy giá trị của nút gửi dữ liệu đi (**submit**) là **Submit Input** nhưng không thấy giá trị của nút được định nghĩa bằng thẻ **< button>**.

8. Ta sẽ thử nghiệm với 2 cách thức gửi dữ liệu của form:
   - Cách thứ nhất là nhấn phím **Enter** khi con trỏ văn bản đặt ở một hộp nhập văn bản (trừ hộp **textarea**): theo cách này, form gửi dữ liệu đi theo cách tương tự như bấm vào nút **Submit Input,** khi đó giá trị của hộp **submit_input** sẽ được gửi kèm.
   - Cách thứ hai là bấm vào nút **submit** thứ 2 (định nghĩa bằng thẻ **<button>**): khi đó giá trị của nút này được gửi đi còn giá trị của nút **Submit Input** sẽ không xuất hiện.

   Ta có thể tận dụng đặc tính này của các nút **submit** để tạo ra các form kết quả phản hồi khác nhau cho cùng một form nhập liệu tùy thuộc người dùng bấm nút nào.

III. Tạo Form sử dụng các thư viện của Django

Bài này sẽ thực hành cách tạo một form sử dụng các thư viện cung cấp bởi **Django**. Form này sẽ có giao diện tương tự như form đã tạo thủ công ở bài thực hành đầu tiên nhưng sẽ cần ít mã lệnh hơn so với cách tạo thủ công. Form này cũng có thêm các tính năng **tự động kiểm tra tính hợp lệ của dữ liệu** và khi ta thay đổi trên form, ta cũng **không phải** thay đổi mã HTML do nó đã được tự động cập nhật lại dựa trên định nghĩa của form.

1. Trong **PyCharm**, tạo một tập tin mới là **forms.py** trong thư mục của app **form_example**.
2. Thêm đoạn mã để **import** thư viện forms của **Django** ở trên đầu tập tin **forms.py**:

```
from django import forms
```

3. Định nghĩa các hộp chọn bằng cách tạo một biến **RADIO_CHOICES** như sau:

```
RADIO_CHOICES = (
    ("Value One", "Value One Display"),
    ("Value Two", "Text For Value Two"),
    ("Value Three", "Value Three's Display Text")
)
```

Ta sẽ dùng biến này để tạo một **ChoiceField** tên là **radio_input** ở bước sau.

4. Định nghĩa một bộ chọn lồng nhau (nhiều cấp) để chọn sách và gán vào biến **BOOK_CHOICES** như sau:

```
BOOK_CHOICES = (
    (
        "Non-Fiction", (
            ("1", "Deep Learning with Keras"),
            ("2", "Web Development with Django")
        )
    ),
    (
        "Fiction", (
            ("3", "Brave New World"),
            ("4", "The Great Gatsby")
        )
    )
)
```

5. Tạo lớp **ExampleForm** kế thừa từ lớp **forms.Form** và thêm các thuộc tính tương ứng với các hộp nhập:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField()
    password_input = forms.CharField(widget=forms.PasswordInput)
    checkbox_on = forms.BooleanField()
    radio_input = forms.ChoiceField(choices=RADIO_CHOICES,
widget=forms.RadioSelect)
    favorite_book = forms.ChoiceField(choices=BOOK_CHOICES)
    books_you_own =
forms.MultipleChoiceField(choices=BOOK_CHOICES)
    text_area = forms.CharField(widget=forms.Textarea)
    integer_input = forms.IntegerField()
```

```
    float_input = forms.FloatField()
    decimal_input = forms.DecimalField()
    email_input = forms.EmailField()
    date_input =
forms.DateField(widget=forms.DateInput(attrs={"type": "date"}))
    hidden_input = forms.CharField(widget=forms.HiddenInput,
initial="Hidden Value")
```

Lưu lại tập tin.

6. Mở tập tin **views.py** của app **form_example**. Ở đầu tập tin thêm dòng **import** lớp **ExampleForm** từ tập tin **forms.py** như sau:

```
from .forms import ExampleForm
```

7. Trong định nghĩa hàm **form_example(),** khởi tạo đối tượng form thuộc lớp **ExampleForm** như sau:

```
    form = ExampleForm()
```

8. Thêm đối tượng form vào biến ngữ cảnh của hàm **render().** Khi đó dòng **return** sẽ có dạng như sau:

```
    return render(request, "form-example.html", {"method":
request.method, "form": form})
```

Lưu lại tập tin **views.py**. Lưu ý **không xóa** bất cứ đoạn mã nào liên quan đến phần in dữ liệu ra giao diện dòng lệnh ở **bài thực hành II.**

9. Mở tập tin **form-example.html** trong thư mục **templates** của app **form_example**. Ta có thể xóa bỏ hầu như tất cả nội dung mã HTML định nghĩa form từ dòng tag template **`{% csrf_token %}`** và các nút **submit**. Kết quả, mã nguồn của tập tin template này như sau:

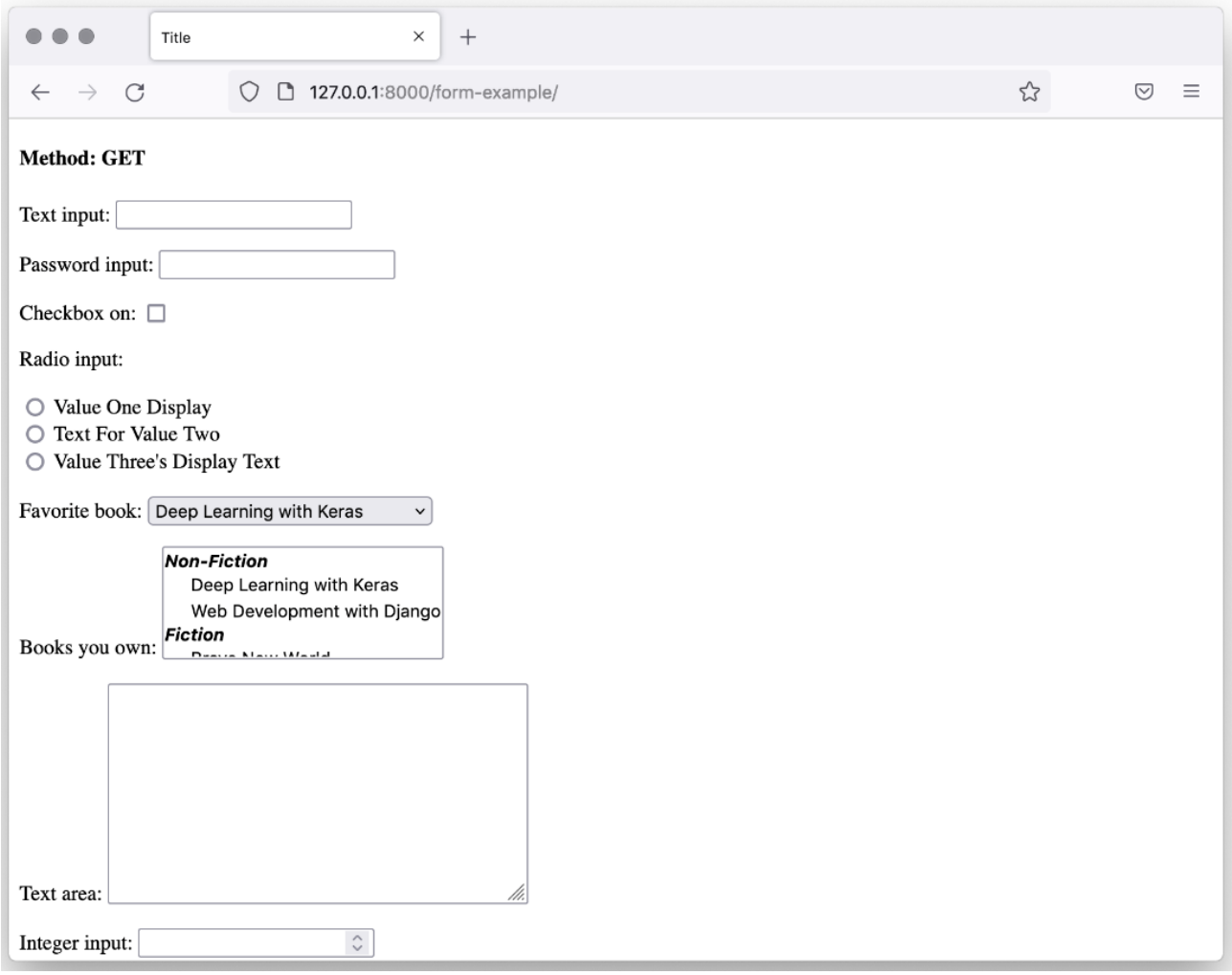
```
<form method="post">
    {% csrf_token %}
    <p>
        <input type="submit" name="submit_input" value="Submit
Input">
    </p>
    <p>
        <button type="submit" name="button_element"
value="Button Element">Button With <strong>Styled</strong>
Text</button>
    </p>
</form>
```

10. Thêm một dòng mã để xuất nội dung Form từ biến form sử dụng phương thức **as_p**. Dòng mã này đặt sau dòng **`{% csrf_token %}`** như sau:

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <p>
        <input type="submit" name="submit_input" value="Submit
Input">
    </p>
```

```
    <p>
        <button type="submit" name="button_element"
value="Button Element">
            Button With <strong>Styled</strong> Text
        </button>
    </p>
</form>
```

11. Khởi chạy máy chủ phát triển Django và truy cập lại vào trang web **form-example** bằng URL http://127.0.0.1:8000/form-example/. Giao diện trang web này như **Hình III-1**.



Hình III-1. Giao diện trang web chứa form sử dụng các thư viện của Django

12. Nhập thử dữ liệu vào một số hộp nhập. Do **Django** đặt tất cả các trường mặc định là bắt buộc phải có dữ liệu nhập vào nên ta phải nhập dữ liệu hoặc chọn tất cả các trường thông tin bao gồm cả đánh dấu vào các **hộp kiểm (checkbox)**. Submit form.
13. Trở lại cửa sổ **PyCharm** và xem trong khung **Debug** hoặc **Run** ở phần dưới cửa sổ. Kết quả xuất ra ở giao diện dòng lệnh sẽ có dạng như **Hình III-2**. Giá trị này tương tự như kết quả ở **bài thực hành II.**

```
csrfmiddlewaretoken: ['aozp1KTShy7e0SbHiig41Qytcfd57BpwpDR3MgyaT1gd9GjOuT5U1sOTVr6JeTIc']
text_input: ['Text']
password_input: ['password']
checkbox_on: ['on']
radio_input: ['Value Two']
favorite_book: ['1']
books_you_own: ['1', '2']
text_area: ['Some text in the text area.']
integer_input: ['10']
float_input: ['10.5']
decimal_input: ['11.5']
email_input: ['user@example.com']
date_input: ['2022-04-20']
hidden_input: ['Hidden Value']
submit_input: ['Submit Input']
```

Hình III-2. Giá trị các hộp nhập của form định nghĩa bằng các thư viện của Django

IV. Kiểm tra tính hợp lệ của dữ liệu nhập từ Form

Bài này sẽ cập nhật lại hàm trong tập tin view để khởi tạo form khác nhau tùy thuộc vào phương thức gửi dữ liệu của gói **HTTPRequest**. Ta cũng sẽ thay đổi form để in ra dữ liệu đã làm sạch thay vì dữ liệu thô của form như các bài thực hành trước.

1. Trong **PyCharm**, mở tập tin **forms.py** trong thư mục của app **form_example**. Thêm một tham số là **max_digits=3** cho hộp nhập thập phân **decimal_input**. Đoạn mã lệnh định nghĩa hộp nhập này sẽ thay đổi như sau:

```
class ExampleForm(forms.Form):
    …
    decimal_input = forms.DecimalField(max_digits=3)
```

   Khi tham số này được thêm vào, ta có thể submit form vì trình duyệt sẽ không biết cách nào để kiểm tra tính hợp lệ của dữ liệu theo quy tắc này nhưng **Django** ở phía máy chủ sẽ kiểm tra.

2. Mở tập tin **views.py** của apps **form_example** và cập nhật lại hàm **form_example** để nếu phương thức gửi dữ liệu là **POST** thì đối tượng form của lớp **ExampleForm** sẽ được khởi tạo với giá trị từ biến **request.POST**. Nếu không thì khởi tạo đối tượng không có tham số. Đoạn mã khởi tạo này sẽ điều chỉnh thành như sau:

```
def form_example(request):
    if request.method == "POST":
        form = ExampleForm(request.POST)
    else:
        form = ExampleForm()
```

3. Tiếp theo, cũng với phương thức gửi là **POST**, ta sẽ kiểm tra xem dữ liệu của form có hợp lệ không bằng phương thức **is_valid.** Nếu dữ liệu là hợp lệ thì sẽ in ra toàn bộ dữ liệu đã làm sạch. Ta thêm một điều kiện khai ở phần khởi tạo biến form để kiểm tra **form.is_valid()** và di chuyển đoạn mã xuất dữ liệu vào trong điều kiện này. Phần xuất dữ liệu từ biến **POST** sẽ điều chỉnh thành như sau:

```
if request.method == "POST":
    form = ExampleForm(request.POST)
    if form.is_valid():
        for name in request.POST:
            print("{}: {}".format(name, request.POST.getlist(name)))
```

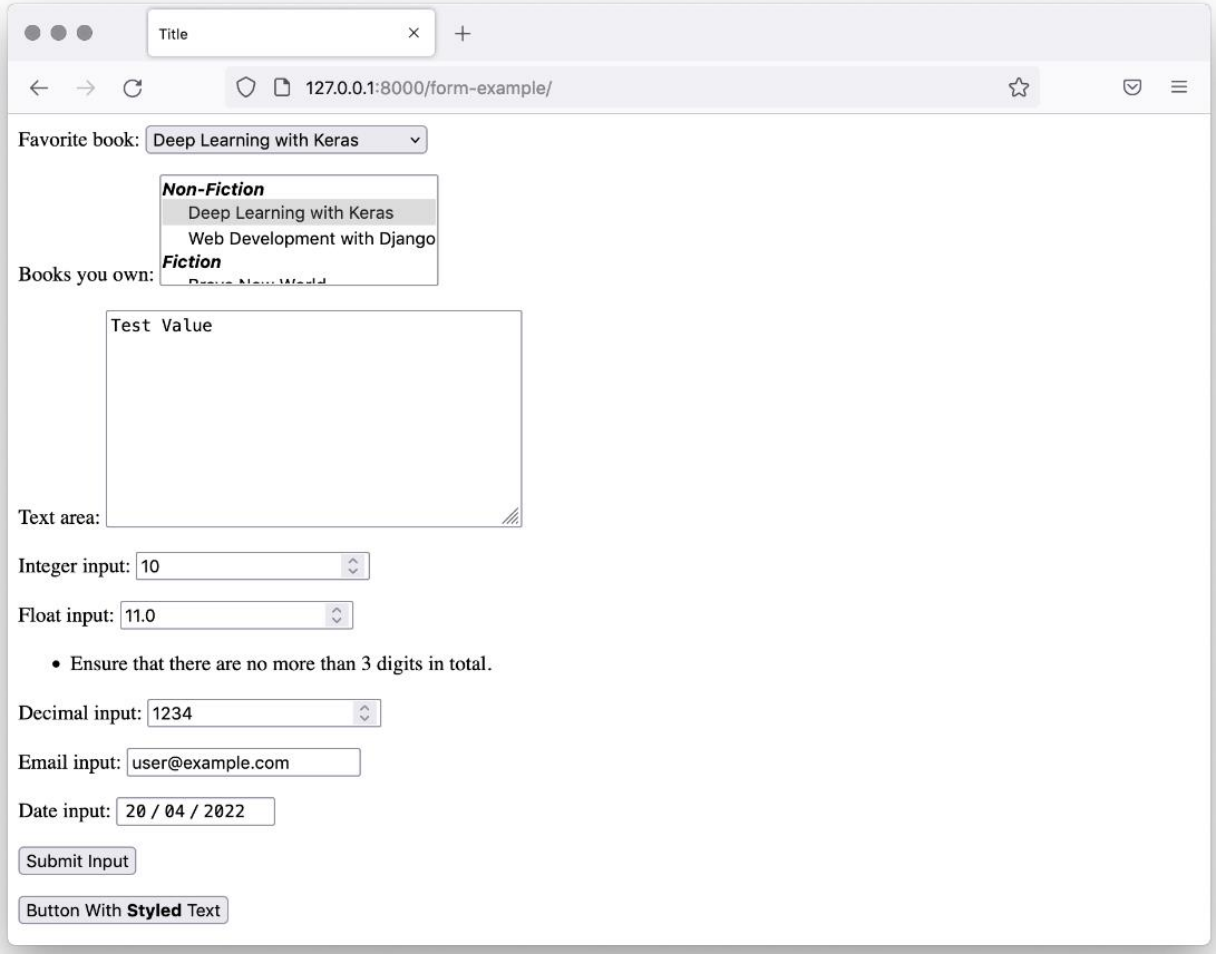
4. Tiếp tục ở phần lặp để xuất dữ liệu, thay vì duyệt qua biến **request.POST** (với dữ liệu xuất ra toàn là kiểu chuỗi văn bản), ta duyệt qua thuộc tính **cleaned_data** chứa dữ liệu đã làm sạch của form. Biến này có kiểu **Dictionary** và chứa các giá trị đã chuyển sang kiểu dữ liệu phù hợp trên **Python**. Hai dòng cuối của đoạn mã ở bước 3 sẽ chuyển thành như sau:

```
        for name, value in form.cleaned_data.items():
            print("{}: ({}) {}".format(name, type(value), value))
```

   Ở đây ta không cần sử dụng phương thức **getlist()** nữa vì Django đã tự chuyển các trường chứa nhiều giá trị sang kiểu **danh sách (list)** rồi.

5. Khởi chạy máy chủ phát triển Django và truy cập đến trang **form-example** bằng URL http://127.0.0.1:8000/form-example/. Giao diện của form không thay đổi so với bài thực

hành trước. Ta nhập dữ liệu vào các hộp nhập và chú ý nhập trên 3 chữ số vào ô **Decimal input** để đảm bảo dữ liệu nhập vào hộp này là không hợp lệ. Submit form và ta sẽ nhận được thông báo lỗi ở hộp nhập **Decimal input** như trong **Hình IV-1**.



Hình IV-1. Thông báo lỗi hiển thị ở trước hộp nhập Decimal input khi submit form

6. Sửa lỗi trên bằng cách chỉ nhập tối đa 3 chữ số vào hộp **Decimal input** và submit lại form. Trở lại cửa sổ **PyCharm** và kiểm tra khung giao diện dòng lệnh **Debug** hoặc **Run**. Kết quả sẽ như **Hình IV-2**.
   Lưu ý: Django đã thực hiện việc chuyển đổi kiểu dữ liệu. Các trường **CharField** đã được chuyển thành kiểu **str**, **BooleanField** chuyển thành kiểu **bool**, **IntergerField**, **FloatField** và **DecimalField** chuyển thành kiểu **int**, **float** và **Decimal** tương ứng. Trường **DateField** thành kiểu **datetime.date,** còn các trường lựa chọn vẫn duy trì kiểu dữ liệu chuỗi như giá trị trong các hộp nhập. Biến **books_you_own** được tự động chuyển thành list của các biến kiểu **str**. Đồng thời, lưu ý là thuộc tính **cleaned_data** chỉ chứa dữ liệu **của form** chứ không chứa các dữ liệu bên ngoài khác (như CSRF token, nút submit,…).

```
text_input: (<class 'str'>) Text
password_input: (<class 'str'>) password
checkbox_on: (<class 'bool'>) True
radio_input: (<class 'str'>) Value One
favorite_book: (<class 'str'>) 1
books_you_own: (<class 'list'>) ['1']
text_area: (<class 'str'>) Test Value
integer_input: (<class 'int'>) 10
float_input: (<class 'float'>) 11.0
decimal_input: (<class 'decimal.Decimal'>) 123
email_input: (<class 'str'>) user@example.com
date_input: (<class 'datetime.date'>) 2022-04-20
hidden_input: (<class 'str'>) Hidden Value
```

Hình IV-2. Dữ liệu xuất ra của form đã được làm sạch

V. Thêm các bộ kiểm tra tính hợp lệ dữ liệu mở rộng

Bài này sẽ thêm và điều chỉnh các quy tắc kiểm tra tính hợp lệ của các trường trong form **ExampleForm**. Sau đó ta sẽ xem những thay đổi này ảnh hưởng thế nào đến hành vi của form trên trình duyệt và khi kiểm tra tính hợp lệ của dữ liệu trên trình chủ web.

1. Trên **PyCharm**, mở tập tin **forms.py** trong thư mục của app **form_example.**

2. Ta sẽ thiết đặt để trường **text_input** chỉ có thể có chiều dài **tối đa 3 ký tự** bằng cách thêm tham số **max_length=3** khi khởi tạo đối tượng **text_input**:

```
text_input = forms.CharField(max_length=3)
```

3. Thiết đặt để trường **password_input** bảo mật hơn bằng cách yêu cầu phải có **tối thiểu 8 ký tự** bằng cách thêm tham số **min_length=8** khi khởi tạo trường này:

```
password_input = forms.CharField(min_length=8,
widget=forms.PasswordInput)
```

4. Người dùng có thể không có cuốn sách nào, vì thế người dùng không bắt buộc phải nhập dữ liệu cho trường **books_you_own**. Ta bỏ ràng buộc này bằng cách thêm tham số **required=False** khi khởi tạo biến **books_you_own.**

```
books_you_own = forms.MultipleChoiceField(required=False,
choices=BOOK_CHOICES)
```

5. Người dùng chỉ có thể nhập một giá trị trong **khoảng từ 1 đến 10** cho trường **integer_input** nên ta thêm các tham số **min_value=1** và **max_value=10** khi khởi tạo biến integer_input:

```
integer_input = forms.IntegerField(min_value=1, max_value=10)
```

6. Cuối cùng, thêm các tham số **max_digits=5** và **decimal_places=3** khi khởi tạo biến decimal_input:

```
decimal_input = forms.DecimalField(max_digits=5,
decimal_places=3)
```

Lưu tập tin.

7. Khởi chạy máy chủ phát triển Django nếu chưa chạy. Ta không cần thiết phải điều chỉnh các tập tin mã nguồn khác khi thay đổi các quy tắc kiểm tra tính hợp lệ do Django đã tự động cập nhật mã HTML sinh ra và quy cách kiểm tra tính hợp lệ. Đây là lợi ích lớn khi sử dụng các thư viện tạo Form của Django. Ta truy cập vào form bằng URL http://127.0.0.1:8000/form-example/. Form này dù giao diện không khác nhưng cách phản ứng khi ta nhập liệu vào đã có nhiều thay đổi - form có thể tự động kiểm tra và hiển thị thông báo lỗi khi dữ liệu nhập không tuân theo các quy tắc mới, như:

- Ta không thể nhập nhiều hơn 3 ký tự cho trường **Text input**
- Nếu nhập ít hơn 8 ký tự cho trường **Password input** mà submit form thì sẽ có thông báo lỗi mật khẩu nhập chưa đủ dài (***Lưu ý: một số trình duyệt không hỗ trợ***).
- Nếu ta không chọn cuốn sách nào trong danh sách **Books you own**, thì trang web không ngăn việc submit form như trước nữa.
- Sử dụng nút tăng giảm giá trị của trường **Integer input**, ta chỉ có thể đặt giá trị trong phạm vi **từ 1 đến 10**. Nếu ta nhập ngoài khoảng này, trình duyệt sẽ báo lỗi.
- Trường **Decimal input** là trường duy nhất không kiểm tra tính hợp lệ của dữ liệu theo các quy tắc của **Django** ngay trên trình duyệt. Ta **phải submit form** mới nhận được thông báo lỗi.

VI. Kiểm tra tính hợp lệ và làm sạch dữ liệu tùy chỉnh

Thực hiện tạo một Form mới tên là **OrderForm** để cho phép người dùng *đặt sách hoặc tạp chí*. Form này cần kiểm tra được các ràng buộc khi nhập liệu như sau:

- Người dùng có thể đặt đến *tối đa 80 cuốn tạp chí và/hoặc 50 cuốn sách*, nhưng tổng số sách và tạp chí *không được vượt quá 100*;
- Người dùng có thể *chọn để nhận email xác nhận đơn đặt hàng* và *nếu họ chọn thì cần nhập địa chỉ email nhận* vào hộp nhập email;
- Người dùng *không nên* nhập email vào hộp nhập email nếu không chọn là nhận xác nhận đơn hàng;
- Để đảm bảo người dùng là thành viên của công ty, địa chỉ email *phải có domain name là của công ty* (ở đây là **example.com**);
- Để đảm bảo tính nhất quán trong ghi địa chỉ email thì email nhập vào cần được chuyển thành dạng *toàn chữ thường (lowercase).*

Tiến hành thực hành theo các bước tại bài tập **Exercise 7.01 - custom clean and validation methods** (Từ **trang 327** của **Giáo trình**) để hoàn thành các yêu cầu trên.

VII. Thêm chỉ dẫn nhập liệu và giá trị mặc định

Bài thực hành này sẽ bổ sung phần chỉ dẫn nhập liệu trong các hộp nhập của form **OrderForm** đã tạo ở bài tập trên, đồng thời truyền một giá trị khởi tạo giả định cho hộp nhập **email** trong form. Giá trị email giả định này về sau sẽ là địa chỉ email của tài khoản người dùng (Sẽ đề cập đến trong **Chương 6**).

1. Trong **PyCharm**, mở tập tin **forms.py** của app **reviews**. Ta cần thêm chỉ dẫn nhập liệu cho các hộp nhập **magazine_count, book_count** và **email** trong form **OrderForm** bằng cách sử dụng **widget**.
   Với hộp nhập **magazine_count**, cần thêm một **widget NumberInput** với giá trị **placeholder** (của biến **attrs**) là **Number of Magazines**. Ta thêm đoạn mã sau:

```
magazine_count = forms.IntegerField(
    min_value=0,
    max_value=80,
    widget=forms.NumberInput(attrs={"placeholder":"Number of
Magazines"}),
)
```

2. Tương tự, thêm một **placeholder** vào hộp nhập **book_count** với giá trị là **Number of Books** bằng đoạn mã sau:

```
book_count = forms.IntegerField(
    min_value=0,
    max_value=50,
    widget=forms.NumberInput(attrs={"placeholder":"Number of
Books"}),
)
```

3. Tiếp theo, với hộp nhập **email**, **widget** sử dụng ở đây là **EmailInput** và nội dung chỉ dẫn là **Your company email address**. Ta thêm đoạn mã sau và lưu lại tập tin:

```
email = forms.EmailField(
    required=False,
    validators=[validate_email_domain],
    widget=forms.EmailInput(attrs={"placeholder":"Your company
email address"}),
)
```

   Lưu ý: Giữ nguyên phần định nghĩa các phương thức clean_email và clean như ở bài thực hành **VI**.

4. Mở tập tin **views.py** của **app form_project**. Trong hàm **form_example**, tạo một biến kiểu **Dictionary** tên là **initial** với một khóa là **email** như sau:

```
initial = {"email": "user@example.com"}
```

5. Ta cũng cần truyền giá trị của biến initial vào tham số **initial** ở 2 đoạn mã khởi tạo đối tượng form của lớp **OrderForm**. Đoạn mã thứ nhất sẽ đổi thành như sau:

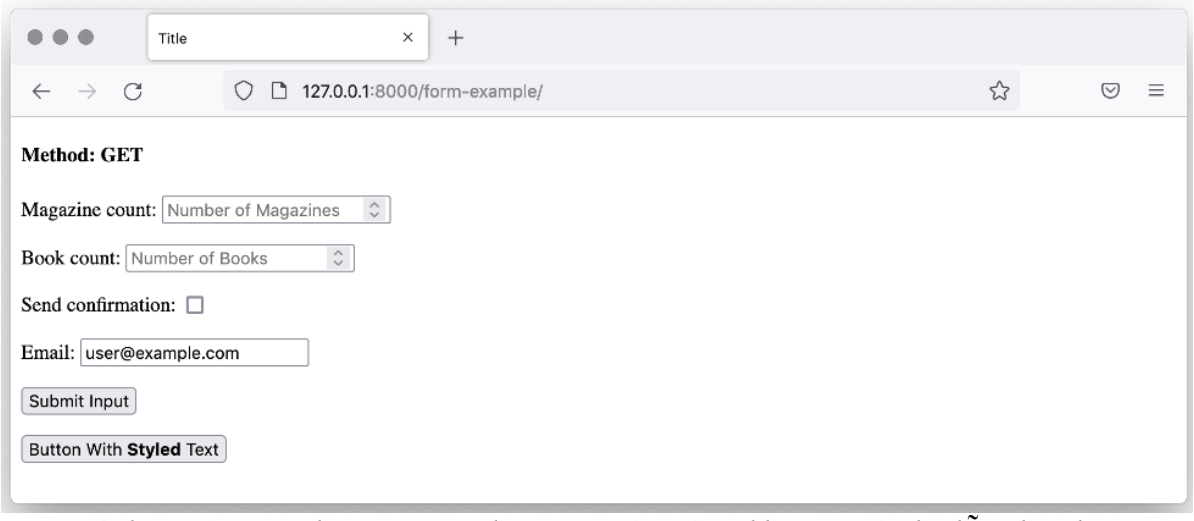
```
form = OrderForm(request.POST, initial=initial)
```

   Đoạn mã thứ hai sẽ thành như sau:

```
form = OrderForm(initial=initial)
```

Lưu lại tập tin **views.py**.

6. Khởi chạy máy chủ phát triển Django và mở trang http://127.0.0.1:8000/form-example/ trên trình duyệt. Giao diện của trang này sẽ như sau:



Hình VII-1. Giao diện trang Order Form với giá trị khởi tạo và chỉ dẫn nhập liệu

VIII. Sử dụng lớp ModelForm để tạo form tạo mới và sửa thông tin Publisher

Bài này sẽ thực hiện ở dự án **Bookr**. Ta sẽ thêm tính năng tạo mới và chỉnh sửa thông tin **Publisher** mà **không cần dùng Django Admin**. Để thực hiện việc này, ta sẽ thêm một **ModelForm** cho Model **Publisher**, **ModelForm** này sẽ được sử dụng trong một hàm View mới. Hàm này sẽ nhận vào một tham số tùy chọn là **pk**, có thể là **ID** của **Publisher** cần *chỉnh sửa thông tin* hoặc là **None** khi cần *tạo mới* một Publisher. Ta sẽ thêm 2 ánh xạ URL cho 2 chức năng này. Khi hoàn tất, người dùng có thể xem và cập nhật thông tin của bất cứ **Publisher** nào dựa vào **ID** truyền qua URL (với đường dẫn **/publishers/<ID>**).

1. Trong **PyCharm**, mở tập tin **forms** của **app Reviews**. Sau dòng **import** thư viện **forms**, ta **import** thêm model **Publisher**:
```
from .models import Publisher
```
2. Tạo một lớp **PublisherForm** kế thừa từ lớp **forms.ModelForm**:
```
class PublisherForm(forms.ModelForm):
```
3. Định nghĩa thuộc tính lớp **Meta** trong **PublisherForm**. Các thuộc tính bắt buộc của lớp Meta gồm **model** (đặt là **Publisher**) và **fields** (đặt là **"__all__"**):
```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = "__all__"
```
Lưu tập tin **forms.py.**

4. Mở tập tin **views.py** của **app Reviews**. Ở phần đầu của tập tin, **import** lớp **PublisherForm**:
```
from .forms import PublisherForm, SearchForm
```
5. Nếu các hàm **get_object_or_404** và **redirect** chưa được **import** từ **lớp django.shortcuts** thì tiến hành import bằng đoạn mã sau:
```
from django.shortcuts import render, get_object_or_404, redirect
```
6. Đồng thời, **import** lớp **Publisher** (*nếu chưa import*) cùng với các lớp Model đã có:
```
from .models import Book, Contributor, Publisher
```
7. Cuối cùng là **import** module **messages**. Module này cho phép ta đăng ký một thông điệp để bảo cho người dùng biết là đang tạo mới hay chỉnh sửa một đối tượng **Publisher**. Thêm đoạn mã sau (nếu chưa có):
```
from django.contrib import messages
```
8. Tạo một hàm **View** mới tên là **publisher_edit**. Hàm này nhận vào **2 tham số** là **request** và **pk** (khóa chính **ID** của đối tượng **Publisher** cần chỉnh sửa thông tin). Đây là tham số tùy chọn, nếu giá trị tham số là **None** (*mặc định*) thì lầ *tạo mới* một **Publisher**:
```
def publisher_edit(request, pk=None):
```
9. Trong hàm **publisher_edit**, ta cần truy xuất dữ liệu của **Publisher** gán cho biến **publisher** khi giá trị của biến **pk khác None**. Nếu **pk là None** thì biến **publisher** nhận giá trị **None**:
```
def publisher_edit(request, pk=None):
    if pk is not None:
        publisher = get_object_or_404(Publisher, pk=pk)
    else:
        publisher = None
```

10. Sau khi gán giá trị cho biến **publisher**, ta xử lý tình huống khi dữ liệu gửi bằng phương thức **POST**. Khởi tạo biến đối tượng form của lớp **PublisherForm** tương tự các bài trước như lưu ý có thêm tham số **instance**. Sau đó, nếu dữ liệu của form là hợp lệ thì lưu vào CSDL dùng phương thức **save().** Phương thức **save()** trả về đối tượng **Publisher** đã cập nhật giá trị mới trong biến **updated_publisher**. Sau đó, đăng ký hiển thị một thông điệp là lưu hoặc cập nhật thành công. Cuối cùng, mở lại trang **publisher_edit** với **ID** của **publisher** đã lưu.

```
def publisher_edit(request, pk=None):
    …
    if request.method == "POST":
        form = PublisherForm(request.POST, instance=publisher)
        if form.is_valid():
            updated_publisher = form.save()
            if publisher is None:
                messages.success(request, "Publisher"{}" was
created.".format(updated_publisher))
            else:
                messages.success(request, "Publisher"{}" was
updated.".format(updated_publisher))
            return redirect("publisher_edit",
updated_publisher.pk)
```

Nếu dữ liệu gửi từ form là **không hợp lệ**, thì đoạn mã lưu dữ liệu sẽ không được thực thi và sẽ chuyển sang gọi hàm **render** (tại Bước 12).

11. Tiếp theo, ta xử lý trường hợp dữ liệu gửi bằng phương thức khác (*không phải là POST*). Khi đó, ta chỉ cần khởi tạo form với tham số **instance** là **publisher**.

```
def publisher_edit(request, pk=None):
    …
    if request.method == "POST":
        …
    else:
        form = PublisherForm(instance=publisher)
```

12. Ở Bước 14, ta sẽ tạo một **template** là **form-example.html** để xuất form nhưng ta có thể gọi hàm **render** đến **template** đó tại bước này:

```
def publisher_edit(request, pk=None):
    …
    return render(request, "form-example.html",{"method":
request.method, "form": form})
```

Lưu tập tin.

13. Mở tập tin **urls.py** trong thư mục **reviews**. Thêm 2 ánh xạ URL, cả 2 ánh xạ này đều truy xuất đến hàm **View publisher_edit**. Một ánh xạ có tham số là **ID** của **Publisher** cần cập nhật thông tin. Ánh xạ còn lại dùng cho tạo mới một Publisher và **không cần truyền ID**.

```
urlpatterns = [
    …
```

```
    path("publishers/<int:pk>/",views.publisher_edit,
name="publisher_edit"),
    path("publishers/new/",views.publisher_edit,
name="publisher_create")
]
```

Lưu lại tập tin.

14. Tạo tập tin template **form-example.html** trong thư mục **templates** của **app reviews** với chức năng **New HTML File** của **PyCharm**. Template này *không cần kế thừa* từ template nào cả nên ta cần có thêm dòng để xuất các thông điệp của thư viện **messages**. Thêm đoạn mã sau vào trong thẻ **<body>:**

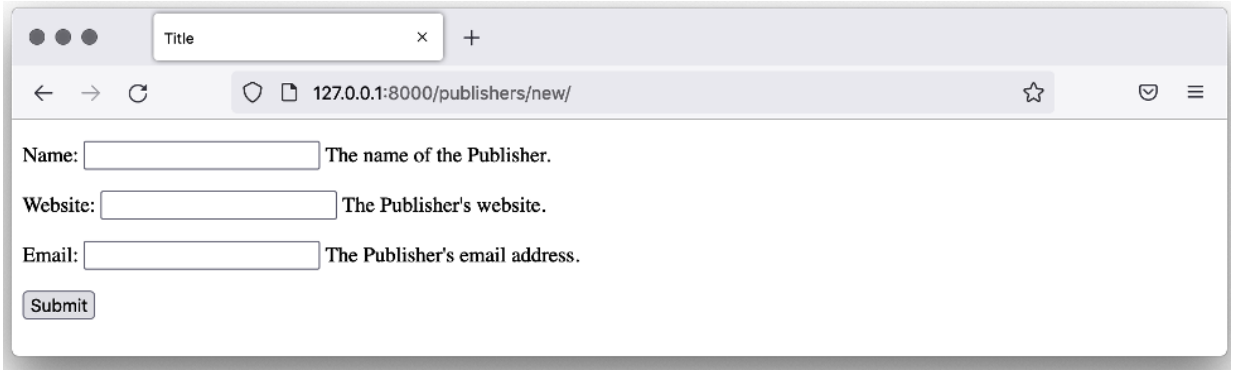
```
{% for message in messages %}
<p><em>{{ message.level_tag|title }}:</em>
{{ message }}</p>
{% endfor %}
```

15. Sau đó, thêm thẻ **<form>** và xuất nội dung của form cùng nút **submit**:

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <p>
        <input type="submit" value="Submit">
    </p>
</form>
```

Lưu tập tin template **form-example.html.**

16. Khởi chạy máy chủ phát triển Django và truy cập đến trang http://127.0.0.1:8000/publishers/new/. Kết quả sẽ như Hình VIII-1.



Hình VIII-1. Form tạo mới Publisher

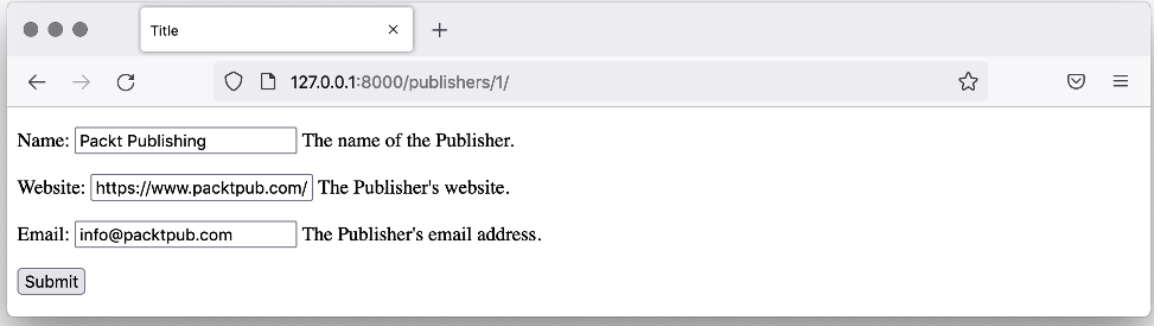
17. Form trên kế thừa các quy tắc kiểm tra tính hợp lệ của dữ liệu từ **Model** nên ta không thể submit form nếu tên quá dài hoặc nhập địa chỉ Website hay Email không hợp lệ. Nhập vào thử các thông tin hợp lệ và submit form, thông tin này sẽ được lưu vào CSDL và ta sẽ nhận được thông điệp thông báo tác vụ đã thực hiện thành công như **Hình VIII-2.**

Lưu ý: Sau khi lưu thì bản ghi dữ liệu đã được lưu vào trong CSDL và **publisher đã có ID**. URL của form kết quả cũng đã có thay đổi, với giá trị ID đã được đưa vào (trong **Hình VIII-2** thì ID publisher vừa lưu có giá trị là **10**). Nếu ta tải lại trang web này thì thông điệp **Lưu thành công** cũng không còn nữa. Đây là do sau khi lưu thì hệ thống đã chuyển hướng

(**redirect**) sang trang mới này. Điều này cũng giúp tránh cho thao tác lưu bị lặp lại nhiều lần nếu người dùng tải lại trang.

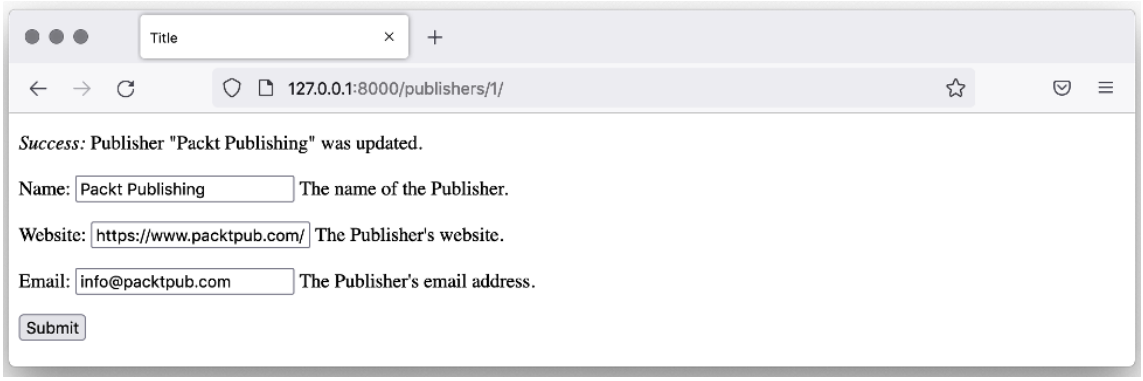
Hình VIII-2. Giao diện form sau khi submit

Nếu đã có các bản ghi khác trong CSDL thì ta cũng có thể đổi giá trị **ID** trên URL để chỉnh sửa bản ghi tương ứng. **Hình VIII-3** hiển thị giao diện chỉnh sửa thông tin của bản ghi có ID là 1.



Hình VIII-3. Giao diện form chỉnh sửa thông tin của Publisher có ID là 1

Thực hiện thay đổi thông tin của publisher và lưu lại. Sau khi lưu xong, trang kết quả sẽ hiển thị thông báo lưu thành công nhưng nội dung lần này khác so với lần trước, như trong Hình VIII-4, do lần này chỉ cập nhật lại thông tin của một bản ghi đã có (không phải là tạo mới như ở bước trước).



Hình VIII-4. Trang kết quả sau khi cập nhật thông tin của publisher có ID là 1